

PUC Minas

FEATURE ENGINEERING E FEATURE STORE

Prof. Victor Sales Silva

victorssbh@hotmail.com

Na aula de hoje...

- Introdução ao feature engineering
- Técnicas de feature engineering

VAMOS COMEÇAR?

Feature engineering e feature store

- Feature engineering é o processo de usar conhecimento dos dados para criar atributos (features) que ajudam os algoritmos de machine learning e inteligência artificial a obter melhor desempenho.
- O conceito originou-se com estatísticos e cientistas de dados que, ao trabalhar com algoritmos mais simples (ex: regressão), perceberam que o sucesso estava menos no modelo e mais em como os dados eram preparados.

Introdução ao feature engineering

- **Por que é importante?**
  - Modelos bons dependem de features informativas; bom *feature engineering* frequentemente traz ganhos maiores que trocar algoritmo.
  - Ajuda a reduzir viés, diminuir ruído e permitir que modelos lineares capturem relações não-lineares.
- **Abordagem prática**
  - **Entendimento dos dados**: tipos, distribuições, missing, cardinalidade.
  - **Hipóteses de domínios**: quais interações/transformações fazem sentido?
  - **Criação**: features derivadas, agregações, janelas temporais.
  - **Validação**: verificar correlação/ganho de informação, evitar vazamento (data leakage).
  - **Automatização**: pipelines (scikit-learn Pipeline, ColumnTransformer) para reprodutibilidade.

Introdução ao feature engineering

- Outras razões importantes pelas quais o processamento de **feature engineering** deve ser analisado são:

  - **Representatividade dos Dados**
  - **Redução de Ruído**
  - **Captura de Relacionamentos Complexos**
  - **Eficiência Computacional**
  - **Interpretabilidade**

Introdução ao feature engineering

Representatividade dos dados

- Muitas vezes, os dados brutos não capturam adequadamente a complexidade do problema. A criação de features adicionais pode ajudar a expressar melhor as informações relevantes contidas nos dados.
  - **Exemplo:** Um conjunto de dados de vendas de produtos que possui os dados por data. Se desejarmos prever as vendas mensais, podemos melhorar a representatividade dos dados criando uma nova feature que indica o mês.

```
import pandas as pd

# Suponha que você tenha um DataFrame chamado 'df' com uma coluna 'data'
df['mes'] = df['data'].dt.month
```

Introdução ao feature engineering

Redução de ruído

- Dados reais frequentemente contêm ruído e outliers que podem afetar negativamente o modelo. O Feature Engineering pode ajudar a mitigar esses problemas, tornando os dados mais limpos e confiáveis.
  - **Exemplo:** Para reduzir o ruído em um conjunto de dados, podemos aplicar uma transformação logarítmica em valores monetários, permitindo lidar com valores extremos e tornar a distribuição de dados mais simétrica.

```
import numpy as np

# Suponha que você tenha uma coluna 'receita' no DataFrame 'df'
df['receita_log'] = np.log1p(df['receita'])
```

Introdução ao feature engineering

Captura de relacionamentos complexos

- Algoritmos de machine learning podem não ser capazes de entender relacionamentos complexos entre features diretamente. A criação de features de interação ou a transformação de features pode ajudar a representar esses relacionamentos de maneira mais compreensível.
  - **Exemplo:** Num conjunto de dados, há informações sobre a idade e o rendimento anual das pessoas. Pode-se criar uma feature que correlacione os dois atributos.

```
df['idade_renda'] = df['idade'] * df['renda_anual']
```

Introdução ao feature engineering

Eficiência computacional

- O número de features em um conjunto de dados pode afetar o tempo de treinamento e os recursos necessários para o modelo. O Feature Engineering pode ajudar a reduzir a dimensionalidade, tornando o processo de treinamento mais eficiente.

Introdução ao feature engineering

Eficiência computacional

- **Exemplo:** Suponha que você tenha um DataFrame com muitas features e deseje selecionar as 10 mais importantes usando o método de Importância das Features de um modelo de árvore de decisão:

```
from sklearn.ensemble import RandomForestClassifier

X = df.drop('target', axis=1)  # X é o conjunto de features
y = df['target']  # y é a variável alvo

# Treine um modelo de Random Forest para obter a importância das features
model = RandomForestClassifier()
model.fit(X, y)

# Selecione as 10 features mais importantes
features_selecionadas = X.columns[model.feature_importances_.argsort()[::-1][:10]]
```

Introdução ao feature engineering

Interpretabilidade

- Algumas features podem ser projetadas para tornar os resultados do modelo mais interpretáveis. Isso é especialmente importante em aplicações em que a explicação do modelo é essencial.
  - Exemplo: Considerando um conjunto de dados com informações de gastos, podemos criar uma feature que indique a porcentagem da renda mensal gasta com dívidas, se estivermos construindo um modelo de crédito.

```
df['porcentagem_divida'] = (df['valor_divida'] / df['renda_mensal']) * 100
```

Introdução ao feature engineering

- Em processos de Feature Engineering, podemos citar algumas técnicas a serem aplicadas para criação de novos atributos num conjunto de dados, tais como:

  - **Transformações lineares**
  - **Transformações não lineares**
  - **Transformações categóricas**
  - **Transformações de seleção**

Introdução ao feature engineering

- Transformações lineares são as mais simples e diretas. Envolvem operações matemáticas que produzem resultados proporcionais aos valores originais.
- Algumas das técnicas de transformações lineares são:

  - **Transformação Logarítmica**
  - **Criação de Features de Interação**
  - **Normalização e Padronização**

Introdução ao feature engineering

- Transformações não lineares são mais complexas, mas podem ser mais eficazes para capturar relações não lineares entre as features e a variável alvo. Envolvem operações matemáticas que não produzem resultados proporcionais aos valores originais.
- Algumas das técnicas de transformações não lineares são:

  - **Extração de Features**
  - **Feature Engineering em NLP**
  - **Feature Engineering em Visão Computacional**
  - **Feature Engineering em Séries Temporais**

Introdução ao feature engineering

- As transformações categóricas são usadas para converter features categóricas em features numéricas. Isso é necessário para que muitos algoritmos de aprendizado de máquina possam ser usados.
- Algumas das técnicas de transformações categóricas:
  - **Codificação de Variáveis Categóricas**

Introdução ao feature engineering

- Transformações de seleção envolvem a escolha de features mais relevantes para um determinado problema.
- Algumas das técnicas de transformações categóricas:

  - **Engenharia de Atributos Temporais**
  - **Seleção de Features**

Técnicas de feature engineering

- Algumas técnicas comuns de feature engineering são:
  - Seleção de features
  - Criação de features derivadas
  - Transformações matemáticas
  - Codificação de variáveis categóricas
  - Normalização e padronização
  - Redução de dimensionalidade
  - Interações entre variáveis
  - Agregações temporais ou por grupo

Técnicas de feature engineering

Seleção de features

- É o processo de escolher, entre todas as variáveis disponíveis, **apenas aquelas que realmente contribuem para o modelo**.
- Com o aumento da complexidade e dimensionalidade dos dados, tornou-se necessário filtrar atributos relevantes, redundantes ou ruidosos. A ideia já era usada em estatística (eliminar outliers e afins).

Técnicas de feature engineering

Seleção de features

- Em muitos problemas, temos dezenas ou até milhares de variáveis (features). Algumas delas não ajudam o modelo a aprender, podendo ser redundantes, irrelevantes ou até atrapalhar. A **seleção de features** busca identificar e manter apenas aquelas que realmente contribuem para a predição.

Técnicas de feature engineering

Seleção de features

- **Objetivos:**
  - Reduzir complexidade do modelo.
  - Aumentar interpretabilidade.
  - Diminuir risco de overfitting.
  - Melhorar tempo de treinamento e inferência.
- **Casos de uso**
  - Bases com muitas variáveis derivadas (sensores, marketing digital, genomics).
  - Problemas em que modelos se tornam lentos ou superajustados devido ao excesso de variáveis.
- **Pontos de atenção**
  - Evitar excluir variáveis sem análise; algumas podem ser pouco informativas sozinhas, mas importantes em conjunto.
  - Seleção deve ser feita **apenas no treino**, nunca no conjunto de teste (evitar data leakage).

Técnicas de feature engineering

Seleção de features

- **Exemplo:**
  - Selecionar as 5 variáveis mais relevantes para prever câncer de mama usando o dataset clássico breast_cancer do scikit-learn.
  - **Resultado esperado:** Uma lista das 5 melhores variáveis segundo a estatística ANOVA F-test.

Técnicas de feature engineering

Seleção de features

```python
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.feature_selection import SelectKBest, f_classif

# Carregar dataset
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

# Selecionar as 5 melhores features
selector = SelectKBest(score_func=f_classif, k=5)
X_selected = selector.fit_transform(X, y)

selected_features = X.columns[selector.get_support()]
print("Features selecionadas:", selected_features.tolist())

```

Técnicas de feature engineering

Criação de features derivadas

- Muitas vezes, os dados brutos não trazem diretamente as informações mais úteis. A criação de features derivadas significa gerar novas variáveis a partir das existentes.
- Nesse cenário, usa-se lógica, manipulação de datas, strings ou relações entre colunas para gerar novas variáveis.
- Pode ser útil em cenários como:
  - Dados brutos não carregam diretamente o padrão desejável
  - Explorar relações não lineares ou agregações temporais.
  - Facilitar a interpretação dos modelos.

Técnicas de feature engineering

Criação de features derivadas

- **Objetivos**
  - Enriquecer o dataset.
  - Capturar relações escondidas entre variáveis.
  - Melhorar performance de modelos preditivos.
- **Casos de uso**
  - E-commerce: criar valor_total = preço * quantidade.
  - Datas: extrair mês, dia da semana, horário.
  - Finanças: calcular indicadores (ROI, margem, etc.).
- **Pontos de atenção**
  - Criar variáveis irrelevantes pode adicionar ruído.
  - Evitar multicolinearidade (quando variáveis são muito parecidas entre si).

Técnicas de feature engineering

Criação de features derivadas

- **Exemplo**
  - Criar a coluna valor_total e extrair informações de tempo de uma coluna de datas.
  - **Resultado esperado:** Novas colunas com informações mais ricas.

Técnicas de feature engineering

Criação de features derivadas

```python
import pandas as pd

df = pd.DataFrame({
    "preco": [10, 20, 15],
    "quantidade": [2, 5, 3],
    "data": pd.to_datetime(["2024-01-01", "2024-03-15", "2024-07-20"])
})

# Criar feature derivada
df["valor_total"] = df["preco"] * df["quantidade"]

# Extrair informações de tempo
df["mes"] = df["data"].dt.month
df["dia_semana"] = df["data"].dt.day_name()

print(df)

```

Técnicas de feature engineering

Transformações matemáticas

- Nem sempre os dados estão em uma escala ou distribuição adequada. Transformações matemáticas ajustam a forma dos dados para torná-los mais manejáveis ou mais próximos de uma distribuição normal.
- As transformações matemáticas podem alterar a escala, forma ou distribuição dos dados numéricos. Aplica-se funções como raiz, exponencial, logaritmo etc. para reduzir assimetrias, tirar outliers ou comprimir faixas de valores.

Técnicas de feature engineering

Transformações matemáticas

- **Objetivos**
  - Reduzir caudas longas (skewness).
  - Destacar variações em dados de alta amplitude.
  - Facilitar o aprendizado de modelos lineares.
- **Casos de uso**
  - Renda, preço, faturamento (dados com cauda longa).
  - Variáveis com crescimento exponencial.
- **Pontos de atenção**
  - Transformações podem dificultar interpretação.
  - Log não funciona para valores negativos ou zero (usar log1p).

Técnicas de feature engineering

Transformações matemáticas

- **Exemplo**
  - Aplicar log e raiz quadrada em valores de renda.
  - **Resultado esperado:** Distribuição mais “comprimida” e próxima do normal.

Técnicas de feature engineering

Transformações matemáticas

```python
import numpy as np
import pandas as pd

df = pd.DataFrame({"renda": [1000, 5000, 20000, 100000]})

df["renda_log"] = np.log1p(df["renda"])
df["renda_sqrt"] = np.sqrt(df["renda"])

print(df)

```

Técnicas de feature engineering

Codificação de variáveis categóricas

- Muitos algoritmos de machine learning não entendem variáveis categóricas diretamente. A codificação transforma categorias em valores numéricos.
- Algumas das técnicas são:
  - One Hot Enconding
  - Label Encoding
  - Target Encoding (Mean Encoding)

Técnicas de feature engineering

Codificação de variáveis categóricas

- One-Hot Encoding é uma técnica usada para converter variáveis categóricas em variáveis binárias (0s e 1s). Ela é usada quando você tem uma variável categórica que não possui uma ordem específica, ou seja, as categorias não têm uma relação ordinal clara.
- A ideia por trás do One-Hot Encoding é criar uma nova coluna binária (0 ou 1) para cada categoria única presente na variável categórica original. Cada nova coluna representa uma categoria e indica se essa categoria está presente ou não em uma observação específica.

Técnicas de feature engineering

Codificação de variáveis categóricas

- Label Encoding é uma técnica usada para converter variáveis categóricas em valores numéricos inteiros. Essa técnica é aplicada quando você tem uma variável categórica com categorias ordinais, ou seja, categorias que têm uma ordem específica, como "baixo," "médio" e "alto" em uma escala de prioridade.
- A ideia por trás do Label Encoding é atribuir a cada categoria um número inteiro único, geralmente começando de 0 e indo até o número de categorias menos 1. Isso permite que o algoritmo de machine learning trabalhe com as categorias de forma numérica.

Técnicas de feature engineering

Codificação de variáveis categóricas

- O Target Encoding, também chamado de Mean Encoding, é uma técnica usada para codificar variáveis categóricas, atribuindo a cada categoria um valor numérico calculado com base na média da variável alvo correspondente. É frequentemente usado quando você deseja incorporar informações da variável alvo diretamente nas variáveis categóricas.
- A ideia por trás do Target Encoding é calcular a média da variável alvo para cada categoria única da variável categórica e, em seguida, atribuir esse valor médio à categoria. Isso significa criar uma única coluna numérica que representa a média da variável alvo para cada categoria.

Técnicas de feature engineering

Codificação de variáveis categóricas

- **Objetivos**
  - Permitir que modelos interpretem categorias.
  - Preservar a informação sem introduzir viés.
- **Casos de uso**
  - Gênero, cidade, produto, categoria de compra.
- **Pontos de atenção**
  - Ordinal encoding pode induzir falsa ordem.
  - One-hot encoding pode gerar muitas colunas (alta dimensionalidade).

Técnicas de feature engineering

Codificação de variáveis categóricas

- **Exemplo**
  - Aplicar OneHotEncoder em uma coluna de cidades.
  - **Resultado esperado:** Cada cidade vira uma coluna binária (0/1).

Técnicas de feature engineering

Codificação de variáveis categóricas

```
from sklearn.preprocessing import OneHotEncoder

df = pd.DataFrame({"cidade": ["SP", "RJ", "BH", "SP", "BH"]})

encoder = OneHotEncoder(sparse=False)
encoded = encoder.fit_transform(df[["cidade"]])
encoded_df = pd.DataFrame(encoded, columns=encoder.get_feature_names_out())

df = pd.concat([df, encoded_df], axis=1)
print(df)

```

Técnicas de feature engineering

Codificação de variáveis categóricas

- **Exemplo**
  - Aplicar TargetEncoder (ou MeanEncoder) na coluna de preço médio de produtos por país.
  - **Resultado esperado:** Preço médio aplicado em um país.

Técnicas de feature engineering

Codificação de variáveis categóricas

PRODUTO	PAÍS	PREÇO MÉDIO
A	EUA	25
B	Canadá	30
C	Brasil	28
D	EUA	20
E	Canadá	35
F	Brasil	40

```python
import pandas as pd

# Crie um DataFrame simulado
data = {'Produto': ['A', 'B', 'C', 'D', 'E', 'F'],
        'País': ['EUA', 'Canadá', 'Brasil', 'EUA', 'Canadá', 'Brasil'],
        'Preço Médio': [25, 30, 28, 20, 35, 40]}

df = pd.DataFrame(data)

# Calcule o Target Encoding para a coluna "País"
mean_encoding = df.groupby('País')['Preço Médio'].mean().to_dict()
df['País_encoded'] = df['País'].map(mean_encoding)
```

Técnicas de feature engineering

Normalização e Padronização

- Diferentes variáveis podem estar em escalas muito diferentes. Ajustar a escala é importante para modelos baseados em distância ou gradiente.

Técnicas de feature engineering

Normalização e Padronização

- A normalização, também conhecida como escala Min-Max, é uma técnica usada para dimensionar os valores de uma variável para um intervalo específico, geralmente entre 0 e 1. Isso é útil quando você deseja que os valores de diferentes variáveis estejam na mesma escala, o que pode ser importante para algoritmos de machine learning sensíveis à escala.

Técnicas de feature engineering

Normalização e Padronização

- A fórmula para a normalização Min-Max é:

$$Znorm = \frac{X - Xmin}{(Xmin - Xmax)}$$

  - **Z_{norm}** é o valor normalizado
  - **X** é o valor original
  - **X_{min}** é o valor mínimo na variável original
  - **X_{max}** é o valor máximo na variável original

Técnicas de feature engineering

Normalização e Padronização

- A padronização, também conhecida como escala Z, é uma técnica usada para transformar uma variável de forma que ela tenha média zero e desvio padrão igual a 1. Isso é útil quando você deseja que os valores estejam centrados em torno de zero e tenham a mesma escala, o que é importante para algoritmos que assumem uma distribuição normal dos dados.

Técnicas de feature engineering

Normalização e Padronização

- A fórmula para a padronização (z-score) é:

$$Z = \frac{X - \mu}{\sigma}$$

  - **Z** é o valor padronizado (z-score)
  - **X** é o valor original
  - μ é a média da variável original
  - **σ** é o desvio padrão da variável original

Técnicas de feature engineering

Normalização e Padronização

- Exemplo:

IDADE	SALÁRIO
25	45000
30	55000
35	60000
40	75000
45	80000

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# Exemplo de dados
data = {'Idade': [25, 30, 35, 40, 45],
        'Salário': [45000, 55000, 60000, 75000, 80000]}

df = pd.DataFrame(data)

# Normalização (Escala Min-Max)
scaler_minmax = MinMaxScaler()
df['Idade_Normalizado'] = scaler_minmax.fit_transform(df[['Idade']])
df['Salário_Normalizado'] = scaler_minmax.fit_transform(df[['Salário']])

# Padronização (Escala Z, z-scores)
scaler_standard = StandardScaler()
df['Idade_Padronizada'] = scaler_standard.fit_transform(df[['Idade']])
df['Salário_Padronizado'] = scaler_standard.fit_transform(df[['Salário']])

print(df)
```

Técnicas de feature engineering

Normalização e Padronização

IDADE	SALÁRIO	IDADE_NORMALIZADO	SALÁRIO_NORMALIZADO	IDADE_PADRONIZADA	SALARIO_PADRONIZADO
25	45000	0.00	0.00	-1.41421	-1.41421
30	55000	0.25	0.25	-0.70711	-0.70711
35	60000	0.50	0.50	0.00	0.00
40	75000	0.75	0.75	0.707107	0,707107
45	80000	1.00	1.00	1.414214	1.414214

Técnicas de feature engineering

Redução de dimensionalidade

- Com muitos atributos, é difícil visualizar e modelar. A redução de dimensionalidade condensa variáveis mantendo a maior parte da informação.

Técnicas de feature engineering

Redução de dimensionalidade

- **Objetivos**
  - Reduzir complexidade.
  - Ajudar na visualização.
  - Remover redundâncias.
- **Casos de uso**
  - Dados de imagem (muitos pixels).
  - Textos (muitos atributos após vetorização).
- **Pontos de atenção**
  - Nem sempre a interpretabilidade é clara.
  - Alguma informação é sempre perdida.

Técnicas de feature engineering

Redução de dimensionalidade

- **Exemplo**
  - Aplicar PCA (técnica estatística **Análise de Componentes Principais**) no dataset de flores Íris.
  - **Resultado esperado:** Reduzir 4 colunas para 2 principais.

Técnicas de feature engineering

Redução de dimensionalidade

```python
from sklearn.decomposition import PCA
from sklearn.datasets import load_iris

iris = load_iris()
X = iris.data

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

print("Forma original:", X.shape)
print("Após PCA:", X_pca.shape)

```

Técnicas de feature engineering

Interações entre variáveis

- Nem sempre as variáveis isoladas carregam toda a informação. A interação entre elas pode ser mais relevante.

Técnicas de feature engineering

Interações entre variáveis

- **Objetivos**
  - Capturar relações ocultas.
  - Criar indicadores compostos.
- **Casos de uso**
  - Saúde: IMC (peso / altura²).
  - Finanças: lucro = receita - custo.
- **Pontos de atenção**
  - Pode gerar multicolinearidade.
  - Interações devem fazer sentido no contexto.

Técnicas de feature engineering

Interações entre variáveis

- **Exemplo**
  - Criar IMC e produto altura*peso.
  - **Resultado esperado:** Novas colunas que representam relações.

Técnicas de feature engineering

Interações entre variáveis

```
df = pd.DataFrame({
    "altura": [1.70, 1.80, 1.65],
    "peso": [65, 85, 55]
})

df["IMC"] = df["peso"] / (df["altura"] ** 2)
df["altura_peso"] = df["altura"] * df["peso"]

print(df)

```

Técnicas de feature engineering

Agregações temporais ou por grupo

- Em conjuntos de dados com componentes temporais, como datas e horários, é comum extrair informações úteis, como dia da semana, mês, trimestre, ou verificar se é um feriado.

Técnicas de feature engineering

Agregações temporais ou por grupo

- **Objetivos**
  - Resumir comportamento de clientes ou períodos.
  - Extrair estatísticas úteis (média, soma, contagem).
- **Casos de uso**
  - Vendas por cliente.
  - Acessos por hora do dia.
- **Pontos de atenção**
  - Agregações podem perder granularidade.
  - É preciso cuidado ao escolher a janela temporal.

Técnicas de feature engineering

Agregações temporais ou por grupo

- **Exemplo**
  - Calcular soma de compras por cliente.
  - **Resultado esperado:** Nova coluna com gasto total do cliente.

Técnicas de feature engineering

Agregações temporais ou por grupo

```python
df = pd.DataFrame({
    "cliente": [1, 1, 2, 2, 3, 3],
    "valor": [100, 200, 50, 70, 300, 400],
    "data": pd.date_range("2024-01-01", periods=6, freq="D")
})

agg = df.groupby("cliente")["valor"].sum().reset_index()
agg.columns = ["cliente", "valor_total"]

df = df.merge(agg, on="cliente")
print(df)

```

DÚVIDAS?

OBRIGADO!